

Compositional reinforcement learning: un approccio categoriale

Filippo Rossi

June 2026

Indice

- 1 Reinforcement Learning
- 2 La categoria **MDP**
- 3 Sottoprocessi e pushout
- 4 Diagrammi a zigzag
- 5 Applicazione: manipolazione robotica
- 6 Osservazioni sulle proprietà dei diagrammi a zigzag

Cos'è il Reinforcement Learning?

Il *reinforcement learning* (RL) è un paradigma in cui un agente impara a comportarsi interagendo con un ambiente al fine di massimizzare un segnale di reward cumulativo.

Tre caratteristiche distinguono il RL dagli altri paradigmi di apprendimento:

- **Ciclo chiuso:** le azioni dell'agente influenzano i suoi input futuri.

Cos'è il Reinforcement Learning?

Il *reinforcement learning* (RL) è un paradigma in cui un agente impara a comportarsi interagendo con un ambiente al fine di massimizzare un segnale di reward cumulativo.

Tre caratteristiche distinguono il RL dagli altri paradigmi di apprendimento:

- **Ciclo chiuso:** le azioni dell'agente influenzano i suoi input futuri.
- **Nessuna supervisione:** l'agente deve scoprire quali azioni producono reward per tentativi.

Cos'è il Reinforcement Learning?

Il *reinforcement learning* (RL) è un paradigma in cui un agente impara a comportarsi interagendo con un ambiente al fine di massimizzare un segnale di reward cumulativo.

Tre caratteristiche distinguono il RL dagli altri paradigmi di apprendimento:

- **Ciclo chiuso:** le azioni dell'agente influenzano i suoi input futuri.
- **Nessuna supervisione:** l'agente deve scoprire quali azioni producono reward per tentativi.
- **Conseguenze ritardate:** le azioni influenzano non solo il reward immediato ma tutti i reward futuri.

Esempi

Vediamo qualche esempio:

- Un giocatore di scacchi che deve decidere di muovere un pezzo: la scelta viene fatta considerando sia i benefici che si possono ottenere da quella mossa ma anche i possibili rischi che può comportare la nuova posizione nel futuro.

Esempi

Vediamo qualche esempio:

- Un giocatore di scacchi che deve decidere di muovere un pezzo: la scelta viene fatta considerando sia i benefici che si possono ottenere da quella mossa ma anche i possibili rischi che può comportare la nuova posizione nel futuro.
- Un robot “riciclatore” che sceglie se entrare in una nuova stanza per cercare altra spazzatura oppure se tornare alla stazione di ricarica.

Concetti chiave

Vediamo ora i concetti chiave del reinforcement learning:

- **Stato** $s \in \mathcal{S}$: la configurazione attuale dell'ambiente.

Concetti chiave

Vediamo ora i concetti chiave del reinforcement learning:

- **Stato** $s \in \mathcal{S}$: la configurazione attuale dell'ambiente.
- **Azione** $a \in \mathcal{A}$: ciò che l'agente può fare.

Concetti chiave

Vediamo ora i concetti chiave del reinforcement learning:

- **Stato** $s \in \mathcal{S}$: la configurazione attuale dell'ambiente.
- **Azione** $a \in \mathcal{A}$: ciò che l'agente può fare.
- **Policy** π : la strategia dell'agente.

Concetti chiave

Vediamo ora i concetti chiave del reinforcement learning:

- **Stato** $s \in \mathcal{S}$: la configurazione attuale dell'ambiente.
- **Azione** $a \in \mathcal{A}$: ciò che l'agente può fare.
- **Policy** π : la strategia dell'agente.
- **Reward** R_t : un segnale numerico che indica quanto è buona la situazione attuale.

Concetti chiave

Vediamo ora i concetti chiave del reinforcement learning:

- **Stato** $s \in \mathcal{S}$: la configurazione attuale dell'ambiente.
- **Azione** $a \in \mathcal{A}$: ciò che l'agente può fare.
- **Policy** π : la strategia dell'agente.
- **Reward** R_t : un segnale numerico che indica quanto è buona la situazione attuale.
- **Funzione valore** $v^\pi(s)$: il reward cumulativo atteso a partire dallo stato s seguendo la policy π .

Policy

Fissiamo degli spazi misurabili $(\mathcal{S}, \mathcal{F}_{\mathcal{S}})$ e $(\mathcal{A}, \mathcal{P}(\mathcal{A}))$.

Definition (Policy stocastica)

Una *policy stocastica* è una sequenza di mappe $\pi_t : \mathcal{S} \rightarrow \mathfrak{P}(\mathcal{A})$, $t \in \mathbb{N}$.

L'idea è che ad ogni s e ad ogni tempo t la mappa $\pi_t(s)$ fornisce una misura di probabilità su $\mathcal{A}$ che si indica con $\pi_t(\cdot|s)$. Quindi, per ogni a la quantità $\pi_t(\{a\}|s)$ viene interpretata come la probabilità che l'agente scelga l'azione a nella posizione s al tempo t .

Reward

Consideriamo una funzione limitata $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$. Quindi, se con S_t e con A_t indichiamo rispettivamente lo stato in cui si trova l'agente al tempo t e l'azione compiuta dall'agente al tempo t allora abbiamo

$$\begin{array}{ccc} R : \mathbb{N} & \rightarrow & \mathbb{R} \\ t & \mapsto & r(S_t, A_t) \end{array}$$

il *signal reward*. Useremo la notazione $R_t := R(t)$.

Funzione valore

Fissata una policy (π_t) e un fattore $\gamma \in (0, 1)$ allora possiamo definire

$$\begin{aligned} v^\pi : \mathcal{S} &\rightarrow \mathbb{R} \\ s &\mapsto \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k R_k \mid S_0 = s \right] \end{aligned}$$

Dove la dipendenza dalla policy π entra in gioco per determinare la strategia con la quale l'agente deve fare le azioni.

Osservazione

Più il parametro γ è alto più uno prende in considerazione il maggior numero di eventi. Più è basso invece più uno prende in considerazione gli eventi che accadono prima temporalmente.

Ordinamento parziale sulle policy

Le funzioni valore inducono un quasi ordinamento parziale sulle policy:

Definition

Una policy π è *minore o uguale* a una policy π' se $v^\pi(s) \leq v^{\pi'}(s)$ per ogni stato $s \in \mathcal{S}$.

La relazione precedente è un quasi ordinamento poichè in generale non vale la proprietà antisimmetrica (si guardi l'ultimo esempio della presentazione). Di conseguenza potrebbe esistere più di un massimo. Ha senso la seguente definizione:

Definition

Una policy che è un massimo per l'ordine definito prima è una *policy ottimale*.

Funzione valore ottimale

Si può dimostrare che due policy ottimali π, η soddisfano

$$v^\pi(s) = v^\eta(s), \quad \forall s \in \mathcal{S}$$

Ha dunque senso definire

$$v_*(s) := \sup_{\pi} v^\pi(s), \quad \forall s \in \mathcal{S},$$

la *optimal state-value function*, che risulta ben posta e raggiunta da ogni policy ottimale.

Processi decisionali di Markov

Definition

Un *MDP* è una 5-upla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \psi, T, R)$:

Processi decisionali di Markov

Definition

Un *MDP* è una 5-upla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \psi, T, R)$:

- $\mathcal{S}$: spazio degli stati (misurabile);

Processi decisionali di Markov

Definition

Un *MDP* è una 5-upla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \psi, T, R)$:

- $\mathcal{S}$: spazio degli stati (misurabile);
- $\mathcal{A}$: spazio delle azioni;

Processi decisionali di Markov

Definition

Un *MDP* è una 5-upla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \psi, T, R)$:

- $\mathcal{S}$: spazio degli stati (misurabile);
- $\mathcal{A}$: spazio delle azioni;
- $\psi : \mathcal{A} \rightarrow \mathcal{S}$: mappa di disponibilità delle azioni (l'azione a è disponibile in $\psi(a)$);

Processi decisionali di Markov

Definition

Un *MDP* è una 5-upla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \psi, T, R)$:

- $\mathcal{S}$: spazio degli stati (misurabile);
- $\mathcal{A}$: spazio delle azioni;
- $\psi : \mathcal{A} \rightarrow \mathcal{S}$: mappa di disponibilità delle azioni (l'azione a è disponibile in $\psi(a)$);
- $T : \mathcal{A} \rightarrow \mathfrak{P}_{\mathcal{S}}$: probabilità di transizione;

Processi decisionali di Markov

Definition

Un *MDP* è una 5-upla $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \psi, T, R)$:

- $\mathcal{S}$: spazio degli stati (misurabile);
- $\mathcal{A}$: spazio delle azioni;
- $\psi : \mathcal{A} \rightarrow \mathcal{S}$: mappa di disponibilità delle azioni (l'azione a è disponibile in $\psi(a)$);
- $T : \mathcal{A} \rightarrow \mathfrak{P}_{\mathcal{S}}$: probabilità di transizione;
- $R : \mathcal{A} \rightarrow \mathbb{R}$: funzione di reward.

Processi decisionali di Markov

Osservazione

- Lo spazio degli stati $\mathcal{S}$ e lo spazio delle azioni $\mathcal{A}$ rappresentano rispettivamente i possibili **stati** e le possibili **azioni** a disposizione dell'agente.

Processi decisionali di Markov

Osservazione

- Lo spazio degli stati $\mathcal{S}$ e lo spazio delle azioni $\mathcal{A}$ rappresentano rispettivamente i possibili **stati** e le possibili **azioni** a disposizione dell'agente.
- La funzione ψ codifica la **disponibilità delle azioni**: un'azione $a \in \mathcal{A}$ è disponibile nello stato $s \in \mathcal{S}$ se e solo se $\psi(a) = s$; l'insieme delle azioni disponibili in s è quindi la controimmagine $\psi^{-1}(s)$.

Processi decisionali di Markov

Osservazione

- Lo spazio degli stati $\mathcal{S}$ e lo spazio delle azioni $\mathcal{A}$ rappresentano rispettivamente i possibili **stati** e le possibili **azioni** a disposizione dell'agente.
- La funzione ψ codifica la **disponibilità delle azioni**: un'azione $a \in \mathcal{A}$ è disponibile nello stato $s \in \mathcal{S}$ se e solo se $\psi(a) = s$; l'insieme delle azioni disponibili in s è quindi la controimmagine $\psi^{-1}(s)$.
- La funzione R assegna una **ricompensa numerica** a ogni azione, definendo il segnale di reward.

Processi decisionali di Markov

Osservazione

- Lo spazio degli stati $\mathcal{S}$ e lo spazio delle azioni $\mathcal{A}$ rappresentano rispettivamente i possibili **stati** e le possibili **azioni** a disposizione dell'agente.
- La funzione ψ codifica la **disponibilità delle azioni**: un'azione $a \in \mathcal{A}$ è disponibile nello stato $s \in \mathcal{S}$ se e solo se $\psi(a) = s$; l'insieme delle azioni disponibili in s è quindi la controimmagine $\psi^{-1}(s)$.
- La funzione R assegna una **ricompensa numerica** a ogni azione, definendo il segnale di reward.
- Le probabilità di transizione T codificano la **dinamica stocastica** dell'ambiente: data un'azione $a \in \mathcal{A}$ disponibile nello stato $s = \psi(a)$, la misura di probabilità $T(a) \in \mathfrak{P}_{\mathcal{S}}$ descrive la distribuzione sugli stati successivi $s' \in \mathcal{S}$ raggiunti dopo aver eseguito a .

Morfismi in MDP

Gli MDP formano una **categoria MDP**, i cui morfismi

$(f, g) : \mathcal{M}_1 \rightarrow \mathcal{M}_2$ sono coppie di mappe compatibili con ψ , T e R . Un morfismo $(f, g) : \mathcal{M}_1 \rightarrow \mathcal{M}_2$ consiste in $f : \mathcal{S}_1 \rightarrow \mathcal{S}_2$ e $g : \mathcal{A}_1 \rightarrow \mathcal{A}_2$ tali che i seguenti diagrammi commutino:

$$\begin{array}{ccc} \mathcal{A}_1 & \xrightarrow{g} & \mathcal{A}_2 \\ \downarrow \psi_1 & & \downarrow \psi_2 \\ \mathcal{S}_1 & \xrightarrow{f} & \mathcal{S}_2 \end{array}$$

$$\begin{array}{ccc} \mathcal{A}_1 & \xrightarrow{g} & \mathcal{A}_2 \\ T_1 \downarrow & & \downarrow T_2 \\ \mathfrak{P}_{\mathcal{S}_1} & \xrightarrow{f_*} & \mathfrak{P}_{\mathcal{S}_2} \end{array}$$

$$\begin{array}{ccc} \mathcal{A}_1 & \xrightarrow{g} & \mathcal{A}_2 \\ & \searrow R_1 & \downarrow R_2 \\ & & \mathbb{R} \end{array}$$

Sottoprocesso e sottoprocesso pieno

Definition (Sottoprocesso e sottoprocesso pieno)

$\mathcal{M}_1$ è un *sottoprocesso* di $\mathcal{M}_2$ se esiste un morfismo $(f, g) : \mathcal{M}_1 \rightarrow \mathcal{M}_2$ tale che sia f che g sono funzioni iniettive. Un sottoprocesso è *pieno* se il diagramma di ψ è cartesiano, ovvero

$$\mathcal{A}_1 \cong \mathcal{S}_1 \times_{\mathcal{S}_2} \mathcal{A}_2 = \{(s, a) \mid f(s) = \psi_2(a)\}.$$

Se $\mathcal{N}$ è un sottoprocesso pieno di $\mathcal{M}$ allora le azioni disponibili in $s \in \mathcal{S}_{\mathcal{N}}$ coincidono con quelle disponibili in $f(s)$ in $\mathcal{M}$: l'agente non può uscire da $\mathcal{N}$ nemmeno usando le azioni di $\mathcal{M}$.

Pushout in **MDP**

Theorem

*Dati morfismi $m_1 : \mathcal{M}_3 \rightarrow \mathcal{M}_1$ e $m_2 : \mathcal{M}_3 \rightarrow \mathcal{M}_2$, il pushout $\mathcal{M}_1 \amalg_{\mathcal{M}_3} \mathcal{M}_2$ esiste in **MDP**:*

$$\begin{array}{ccc}
 \mathcal{M}_3 & \xrightarrow{m_1} & \mathcal{M}_1 \\
 m_2 \downarrow & & \downarrow \\
 \mathcal{M}_2 & \longrightarrow & \mathcal{M}_1 \amalg_{\mathcal{M}_3} \mathcal{M}_2
 \end{array}$$

Corollary

Se $\mathcal{M}_3$ è sottoprocesso di $\mathcal{M}_1$ e $\mathcal{M}_2$, allora $\mathcal{M}_1$ e $\mathcal{M}_2$ sono sottoprocessi di $\mathcal{M}_1 \amalg_{\mathcal{M}_3} \mathcal{M}_2$.

Pushout in MDP

Il teorema visto sopra ci assicura l'esistenza del pushout, ma che forma ha?

Osservazione

Dati tre MDP $\mathcal{M}_1$, $\mathcal{M}_2$ e $\mathcal{M}_3$ allora l'insieme degli stati $\mathcal{S}$ e delle azioni $\mathcal{A}$ del pushout ha la stessa forma del pushout nella categoria **Set**.

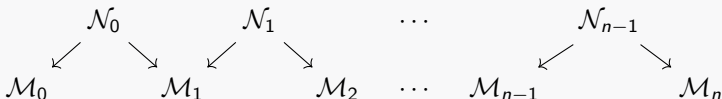
In altre parole vale $\mathcal{A} = (\mathcal{A}_1 \sqcup \mathcal{A}_2) / \sim$ dove $x \sim y$ se e solo se $x \in \mathcal{A}_1$, $y \in \mathcal{A}_2$ ed esiste $z \in \mathcal{A}_3$ tale che $g_1(z) = x$ e $g_2(z) = y$ (analogo per $\mathcal{S}$).

$$\begin{array}{ccc}
 \mathcal{A}_3 & \xrightarrow{g_1} & \mathcal{A}_1 \\
 g_2 \downarrow & & \downarrow \pi_{\sim} \circ i_1 \\
 \mathcal{A}_2 & \xrightarrow{\pi_{\sim} \circ i_2} & \mathcal{A}_1 \amalg_{\mathcal{A}_3} \mathcal{A}_2
 \end{array}$$

Diagrammi a zigzag e MDP composti

Definition (Diagramma a zigzag e MDP composto)

Un *diagramma a zigzag* è un diagramma in **MDP** della forma



dove $\mathcal{N}_i$ è sottoprocesso di $\mathcal{M}_i$ e $\mathcal{M}_{i+1}$. L'*MDP composto* è definito induttivamente:

$$\mathcal{C}_0 := \mathcal{M}_0, \quad \mathcal{C}_k := \mathcal{C}_{k-1} \coprod_{\mathcal{N}_{k-1}} \mathcal{M}_k.$$

Diagrammi a zigzag e MDP composti

Perché introdurre le definizioni di prima? L'idea è la seguente: considerare gli $\mathcal{M}_i$ come gli ambienti e gli $\mathcal{N}_i$ come gli stati “obiettivi” all'interno dell'ambiente $\mathcal{M}_i$. L'MDP composto $\mathcal{C}_n$ viene costruito con l'idea di racchiudere in un unico oggetto tutti gli ambienti. Formalmente, questo si traduce in:

Theorem

$$\mathcal{C}_n = \text{colim}(D).$$

Diagrammi forward-moving

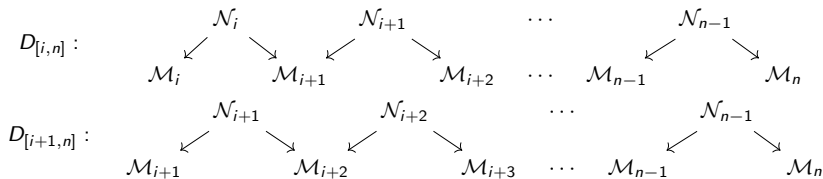
Definition (Forward-moving)

Un diagramma a zigzag è *forward-moving* se ogni $\mathcal{N}_i$ è un sottoprocesso **pieno** di $\mathcal{M}_i$.

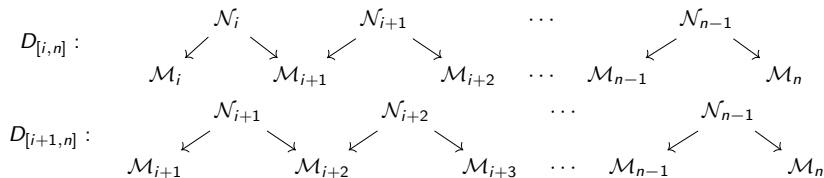
Il seguente lemma giustifica il nome della definizione:

Lemma

Se D è *forward-moving*, allora $\mathcal{C}_{[i+1,n]}$ è un sottoprocesso pieno di $\mathcal{C}_{[i,n]}$.



Diagrammi forward-moving



Se $\mathcal{N}_i$ è sottoprocesso di $\mathcal{M}_i$ e $\mathcal{C}_0 := \mathcal{M}_0$, $\mathcal{C}_k := \mathcal{C}_{k-1} \coprod_{\mathcal{N}_{k-1}} \mathcal{M}_k$ allora dire che $\mathcal{C}_{[i+1,n]}$ è un sottoprocesso pieno di $\mathcal{C}_{[i,n]}$ vuol dire che una volta che l'agente è in uno stato s di $\mathcal{C}_{[i+1,n]}$ e lo guardo in $\mathcal{C}_{[i,n]}$ (ovvero che ha completato la i -esima task) questo non potrà più andare in uno stato fuori da $\mathcal{C}_{[i+1,n]}$. Quindi una volta completata una task, l'agente dovrà procedere per forza all'interno dell'ambiente della prossima task.

Monotonia

La seconda ipotesi di cui avremo bisogno è la seguente.

Definition

Consideriamo un diagramma a zigzag come quelli introdotti in precedenza. Diremo che la funzione valore ottimale $v_*^{C_n}$ è *monotona* se, per ogni $i \in \{1, \dots, n\}$ e per ogni $s \in \mathcal{S}_i$, le quantità

$$\sum_{s' \in \mathcal{S}_i} T(a)(s') (R_i(a) + \gamma v_*^{C_n}(s'))$$

e

$$\sum_{s' \in \mathcal{S}_i} T(a)(s') (R_i(a) + \gamma v_*^{C_{[i,n]}}(s'))$$

sono massimizzate esattamente dalle stesse azioni $a \in (\mathcal{A}_i)_s$.

Monotonia

L'azione ottimale in uno stato $s \in \mathcal{S}_i$ è quella che massimizza

$$\sum_{s' \in \mathcal{S}_i} T(a)(s') (R_i(a) + \gamma v_*(s')).$$

La definizione di monotonia confronta questa scelta in due MDP:

$$\mathcal{C}_n \quad \text{e} \quad \mathcal{C}_{[i,n]}$$

dove $\mathcal{C}_{[i,n]}$ si ottiene da $\mathcal{C}_n$ eliminando i task $\mathcal{M}_0, \dots, \mathcal{M}_{i-1}$. Quindi si confrontano le quantità:

$$\sum_{s' \in \mathcal{S}_i} T(a)(s') (R_i(a) + \gamma v_*^{\mathcal{C}_n}(s')) \quad \sum_{s' \in \mathcal{S}_i} T(a)(s') (R_i(a) + \gamma v_*^{\mathcal{C}_{[i,n]}}(s'))$$

Monotonia

Utilizzando il teorema di Bellman:

Theorem

Per ogni stato $s \in \mathcal{S}$ di un MDP $\mathcal{M}$, la funzione di valore ottimale soddisfa

$$v_*(s) = \max_{a \in \mathcal{A}_s} \sum_{s' \in \mathcal{S}} T(a)(s') [R(a) + \gamma v_*(s')]$$

possiamo osservare che l'azione ottimale è precisamente l'azione che realizza il valore ottimo nei due MDP. Visto che la definizione richiede che

$$\operatorname{argmax} \left(\sum_{s' \in \mathcal{S}} T(a)(s') [R(a) + \gamma v_*^{\mathcal{C}_n}(s')] \right) = \operatorname{argmax} \left(\sum_{s' \in \mathcal{S}} T(a)(s') [R(a) + \gamma v_*^{\mathcal{C}_{[i,n]}}(s')] \right)$$

Quindi il valore della funzione ottimale di valutazione in $\mathcal{C}_n$ non dipende dai task che la precedono.

Teorema principale

Theorem (Teorema principale)

Sia $\mathcal{C}_n$ un diagramma a zigzag che soddisfa le seguenti ipotesi:

- *è forward-moving;*
- *la sua funzione valore ottimale è monotona.*

Allora la policy ottenuta seguendo, su ciascuna componente $\mathcal{M}_i$, la corrispondente policy ottimale π_i è una policy ottimale per l'MDP composto $\mathcal{C}_n$.

Il task di manipolazione robotica

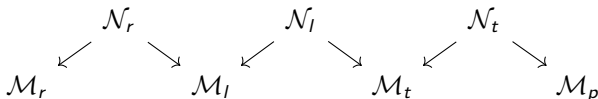
Applichiamo il framework compositazionale a un task di manipolazione robotica [BSZ⁺25]: spostare un oggetto da una posizione a un'altra.

Quattro MDP di sotto-task:

- $\mathcal{M}_r$: **Raggiungimento** — avvicinare il gripper all'oggetto.
- $\mathcal{M}_l$: **Sollevamento** — sollevare l'oggetto dal tavolo.
- $\mathcal{M}_t$: **Trasporto** — portare l'oggetto dall'altra parte del tavolo.
- $\mathcal{M}_p$: **Posizionamento** — posare l'oggetto nella posizione goal g .

Il diagramma a zigzag

Il task completo è codificato dal seguente diagramma a zigzag:



- $\mathcal{N}_r$: gripper entro 1 cm dall'oggetto, $\|\mathbf{p}_r - \mathbf{p}_o\|_2 < 0.01$.
- $\mathcal{N}_l$: altezza oggetto sopra soglia, $z_o > h$.
- $\mathcal{N}_t$: robot ha superato la linea centrale, $y_r > 0.03$.

Seguendo [BSZ⁺25], il diagramma è forward-moving e la condizione di monotonicità è soddisfatta. Per il Teorema principale, componendo le policy ottimali dei sotto-task si ottiene una policy ottimale per il task completo.

Risultati sperimentali

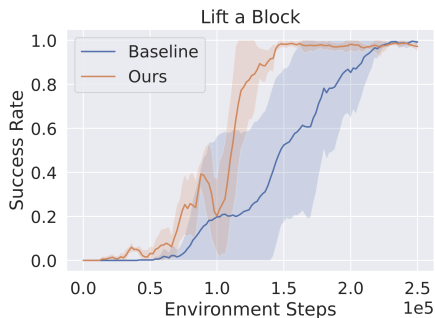


Figure: Tasso di successo in funzione dei passi di training [BSZ⁺25].

- **Metodo compositazionale:** 100% di successo dopo $\sim 150k$ passi.
- **Baseline:** 100% di successo dopo $\sim 225k$ passi.

Rendere un diagramma forward-moving

Osservazione

Dato un diagramma a zigzag qualsiasi, è sempre possibile ottenerne uno **forward-moving** modificando gli $\mathcal{M}_i$, non gli $\mathcal{N}_i$.

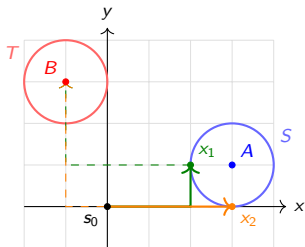
La strategia è restringere $\mathcal{M}_i$ togliendo esattamente le azioni che, partendo da uno stato di $\mathcal{N}_i$, non sono già azioni di $\mathcal{N}_i$. Così $\mathcal{N}_i$ diventa un sottoprocesso **pieno** di $\widehat{\mathcal{M}}_i$.

Attenzione

Non è gratis: restringere $\mathcal{M}_i$ toglie all'agente azioni che aveva originariamente a disposizione negli stati di $\mathcal{N}_i$, quindi il composito $\mathcal{C}_n$ risultante può cambiare.

Fallimento della monotonia

Consideriamo un agente che si muove su una griglia $\mathbb{Z}^2$ che parte dall'origine.



Quindi consideriamo :

- $\mathcal{M}_0$, l'MDP che assegna reward positivo se l'agente è nella palla A altrimenti un reward negativo.
- $\mathcal{N}_0$, l'MDP che ha come stati gli elementi della palla A.
- $\mathcal{M}_1$, l'MDP che assegna reward positivo se l'agente è nella palla B altrimenti un reward negativo.

Abbiamo allora $\mathcal{C}_0 = \mathcal{M}_0$ e $\mathcal{C}_1 = \mathcal{M}_0 \coprod_{\mathcal{N}_0} \mathcal{M}_1$, quindi segue che

$$\operatorname{argmax}_{\mathcal{M}_0} \neq \operatorname{argmax}_{\mathcal{C}_1}$$

Infatti, le azioni che portano a x_1 sono più efficienti (per il secondo task) rispetto alle azioni che portano in x_2 .

Riferimenti



Georgios Bakirtzis, Michail Savvas, Ruihan Zhao, Sandeep Chinchalic, and Ufuk Topcu.

Reduce, reuse, recycle: Categories for compositional reinforcement learning, 2025.

Article, available at <https://arxiv.org/abs/2408.13376>.



Michail Savvas Georgios Bakirtzis and Ufuk Topcu.

Categorical semantics of compositional reinforcement learning, 2025.

Article, available at <https://arxiv.org/abs/2208.13687>.



Richard S. Sutton and Andrew G. Barto.

Reinforcement learning: An introduction, 2014-2015.

Second edition in progress, available at <https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf>.